Introdução à Orientação a Objetos

Aula 11 — Herança (parte 2), polimorfismo e classes abstratas

Programação Orientada a Objetos

Semana 11

Sumário

1. Abertura
2. Etapa 1: sobrescrita
3. Etapa 2: `Object`, `equals` e `String`
4. Etapa 3: polimorfismo e casting
5. Etapa 4: classe abstrata
6. Fechamento

Onde paramos

Na aula 10 criamos a hierarquia da livraria:

- `Ebook`, `LivroFisico` e `Audiolivro` **herdam** de `Livro`
- cada filha chama `super(autor)` no construtor
- o `Livro` **tem uma** `Editora` (composição)

E deixamos uma pergunta no ar:

```
1 Ebook ebook = new Ebook(autor);
2 ebook.setValor(30.00);
3 System.out.println(ebook.aplicaDescontoDe(0.3));    // true
```

O e-book **herdou** o desconto de 30%. Mas e-book deveria aceitar **no máximo 15%**.

Hoje

Quatro perguntas, uma etapa para cada uma:

1. Como uma filha **muda** uma regra herdada? *sobrescrita*
2. O que **todas** as classes herdam, mesmo sem `extends`? `Object`
3. Posso tratar um `Ebook` como um `Livro`? *polimorfismo*
4. Faz sentido existir um `Livro` que não é de nenhum tipo? *abstrata*

Em cada etapa: um pouco de conceito, e depois **em duplas**, no código da livraria.

Como vamos trabalhar

Cada etapa segue o mesmo ciclo:

1. o professor apresenta o **conceito** (10 a 15 min)
2. a dupla **implementa** — quem digita troca a cada etapa
3. **checkpoint**: rode o teste e compare com a saída esperada
4. saída bateu? siga para a próxima etapa

- Travou? Tente por 5 minutos, releia as pistas, depois chame o professor
- O enunciado completo está na lista (PDF), com os mesmos testes e saídas

Mapa da aula

Etapa	O que fazemos	Tempo
1	Sobrescrita: `@Override`, `super`, `toString`	45 min
2	`Object`, `equals` e `String`	45 min
3	Polimorfismo e casting no `CarrinhoDeCompras`	45 min
4	`Livro` abstrato e `getFormato()`	45 min
	Fechamento	20 min

O projeto de partida

Baixe o `.zip` no Moodle e abra a pasta `src` na IDE. É o projeto da aula 10:

Pacote	Classes
`br.edu.ifsp.livraria`	`Autor`, `Editora`, `CarrinhoDeCompras`, `Catalogo`
`br.edu.ifsp.livraria.produtos`	`Livro`, `Ebook`, `LivroFisico`, `Audiolivro`
`br.edu.ifsp.livraria.testes`	`CadastroDeLivros` (o `main`), `TiposDeLivro`

O `main` de `CadastroDeLivros` deve imprimir:

```
Livros cadastrados: 2
Total do carrinho: 148.9
```

Daqui em diante, cada etapa **acrescenta** código ao final do `main`.

Reescrevendo métodos da superclasse

A filha pode **reescrever** (*sobrescrever*) um método herdado, com a **mesma assinatura**:

```
1  public class Ebook extends Livro {
2      public boolean aplicaDescontoDe(double porcentagem) {
3          if (porcentagem > 0.15) {
4              return false;
5          }
6          double desconto = this.getValor() * porcentagem;
7          this.setValor(this.getValor() - desconto);
8          return true;
9      }
10 }
```

- A JVM procura o método **primeiro no objeto instanciado**; só se não achar, sobe para a superclasse
- `valor` é `private` em `Livro`: a filha usa `getValor`/`setValor`

@Override: o compilador confere

```java
@Override
public boolean aplicaDescontoDe(double porcentagem) { ... }
```

- É uma **anotação**: opcional, mas vale sempre usar
- Se o método **não existir** na superclasse, o código não compila

Errou o nome (`aplicaDesconto`) ou o parâmetro? O `javac` avisa:

```
Ebook.java:22: error: method does not override or implement a method from a
    supertype
    @Override
    ^
```

this ou super?

Por que reescrever tudo? Podemos **delegar** para a regra da superclasse:

```
@Override
public boolean aplicaDescontoDe(double porcentagem) {
    if (porcentagem > DESCONTO_MAXIMO_EBOOK) {
        System.out.println("Desconto acima do permitido para ebook");
        return false;
    }
    return super.aplicaDescontoDe(porcentagem);
}
```

- `super.metodo()` chama o método **da classe pai**, evitando repetir a lógica
- Com `this.aplicaDescontoDe(...)` o método chamaria **a si mesmo**: laço infinito
- `DESCONTO_MAXIMO_EBOOK` é uma constante (`static final`) da classe `Ebook`

Toda classe herda de `Object`

```
1 System.out.println(ebook);
```

```
br.edu.ifsp.livraria.produtos.Ebook@64cee07
```

- Sem `extends`, o Java coloca `extends Object`: **toda classe é um** `Object`, direta ou indiretamente
- `println(objeto)` chama o método `toString()`, herdado de `Object`
- O `toString` padrão imprime o nome da classe e um número (muda a cada execução)

$$\texttt{Object} \leftarrow \texttt{Livro} \leftarrow \texttt{Ebook}$$

Sobrescrevendo o `toString`, nós decidimos **como o objeto vira texto**.

toString com super

O `mostrarDetalhes` da `Livro` vira `toString`, que **devolve** o texto em vez de imprimir:

```java
// em Livro
@Override
public String toString() {
    String texto = "Nome: " + nome + "\n" + "ISBN: " + isbn + "\n"
            + "Valor: " + String.format("R$ %.2f", valor);
    // ... data, autor, editora
    return texto;
}

public void mostrarDetalhes() {
    System.out.println(this.toString());
}

// em Ebook: acrescenta uma linha ao texto da Livro
@Override
public String toString() {
    return super.toString() + "\nMarca d'agua: " + waterMark;
}
```

Etapa 1: objetivo

1. `Ebook` sobrescreve `aplicaDescontoDe`: acima de **15%** imprime `Desconto acima do permitido para ebook` e devolve `false`; senão, **delega** ao `super`. O limite é a constante `DESCONTO_MAXIMO_EBOOK`
2. `Livro`: crie `toString()` com o conteúdo do `mostrarDetalhes` (linhas separadas por `\n`, **sem** o `--"` final); `mostrarDetalhes()` passa a só imprimir o `toString()`
3. `Ebook`, `LivroFisico` e `Audiolivro` sobrescrevem `toString()`: o texto de `super.toString()` mais uma linha
   - `Marca d'agua: ...`, `Taxa de impressao: R$ 3,00`, `Duracao: 6h35min`

Etapa 1: pistas

- Marque os métodos reescritos com `@Override`
- O `toString` usa `+` e `"\n"`; o valor sai com `String.format("R$ %.2f", valor)`
- Atributos `private` da `Livro` não aparecem nas filhas: use os *getters*
- `getTaxaImpressao()` e `getDuracaoFormatada()` já existem
- Se a data ou a editora forem `null`, essas linhas **não** entram no texto

Etapa 1: checkpoint (1/2)

Acrescente ao final do `main`:

```
System.out.println("--- Etapa 1 ---");
Autor aniche = new Autor("Mauricio Aniche");

Ebook ebook = new Ebook(aniche);
ebook.setNome("Test-Driven Development");
ebook.setIsbn("978-85-5519-024-8");
ebook.setValor(30.00);
ebook.setWaterMark("aluno@ifsp.edu.br");

LivroFisico fisico = new LivroFisico(aniche);
fisico.setNome("Test-Driven Development");
fisico.setIsbn("978-85-5519-023-1");
fisico.setValor(60.00);
```

Etapa 1: checkpoint (2/2)

```
Audiolivro audio = new Audiolivro(aniche, 395);
audio.setNome("Test-Driven Development");
audio.setIsbn("978-85-5519-025-5");
audio.setValor(40.00);

ebook.mostrarDetalhes();
System.out.println();
fisico.mostrarDetalhes();
System.out.println();
audio.mostrarDetalhes();
System.out.println(ebook.aplicaDescontoDe(0.3));
System.out.println(ebook.aplicaDescontoDe(0.1));
System.out.println(ebook.getValor());
```

Etapa 1: saída esperada

```
--- Etapa 1 ---
Nome: TEST-DRIVEN DEVELOPMENT
ISBN: 9788555190248
Valor: R$ 30,00
Autor: Mauricio Aniche
Marca d'agua: aluno@ifsp.edu.br

Nome: TEST-DRIVEN DEVELOPMENT
ISBN: 9788555190231
Valor: R$ 60,00
Autor: Mauricio Aniche
Taxa de impressao: R$ 3,00

Nome: TEST-DRIVEN DEVELOPMENT
ISBN: 9788555190255
Valor: R$ 40,00
Autor: Mauricio Aniche
Duracao: 6h35min
Desconto acima do permitido para ebook
false
true
27.0
```

Comparando objetos: == ou equals?

```
1 Autor a1 = new Autor("Rodrigo Turini");
2 Autor a2 = new Autor("Rodrigo Turini");
3 System.out.println(a1 == a2);          // false
4 System.out.println(a1.equals(a2));     // false (!)
```

- == compara **referências**: são o mesmo objeto na memória?
- O `equals` herdado de `Object` **também** faz ==
- O Java não sabe quando dois autores são "iguais": **nós ensinamos**, sobrescrevendo o `equals`

A String já faz isso

```java
String a = "livro";
String b = new String("livro");
System.out.println(a == b);          // false
System.out.println(a.equals(b));     // true
```

- `String` **herda** de `Object` e **sobrescreve** o `equals`: compara **o texto**, caractere por caractere
- Fecha a armadilha da aula 9: texto se compara com `equals`, **nunca** com ==
- (`"java"== "java"` dá `true` só por causa do *String pool*, uma otimização da JVM — não conte com isso)

Ensinando o equals ao Livro

Critério da livraria: dois livros são o mesmo livro se têm o **mesmo ISBN**.

```java
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (!(obj instanceof Livro)) {
        return false;
    }
    Livro outro = (Livro) obj;
    return this.isbn != null && this.isbn.equals(outro.isbn);
}
```

- O parâmetro é `Object`: serve para comparar com **qualquer** coisa
- `instanceof` evita o `ClassCastException`; o *cast* `(Livro)` vem na etapa 3

Quem usa o equals?

- `lista.contains(x)` e `lista.remove(x)` comparam com `equals`
- Por isso o `carrinho.remove(Livro)` da aula 9 passa a remover **o livro de mesmo ISBN**, mesmo que seja outro objeto
- `getClass().getSimpleName()` mostra o nome da classe, sem o pacote

Atenção: o `equals` tem um "irmão" que sempre o acompanha, o `hashCode`. Vamos tratar dele com as coleções; por ora, só o `equals`.

Etapa 2: objetivo

1. `Livro` sobrescreve `equals(Object)`: iguais se têm o **mesmo ISBN**
   - o mesmo objeto é sempre igual a si mesmo
   - outro tipo de objeto, `null` ou livro **sem** ISBN: `false`
2. Sem alterar o `CarrinhoDeCompras`, observe o `remove(Livro)` usando o seu `equals`
3. Compare duas `String` com `==` e com `equals`

Etapa 2: pistas

- A assinatura é `public boolean equals(Object obj)` — com `@Override`
- Se escrever `equals(Livro obj)`, é **sobrecarga**, não sobrescrita: o `@Override` acusa o erro
- Ordem: mesmo objeto → `instanceof` → *cast* → comparar o ISBN
- `isbn` é `String`: compare com `equals`, e cuide do `null`
- Dentro da classe `Livro`, `outro.isbn` é acessível

Etapa 2: checkpoint

```
System.out.println("--- Etapa 2 ---");
LivroFisico copia = new LivroFisico(aniche);
copia.setIsbn("978-85-5519-023-1");
System.out.println(fisico == copia);
System.out.println(fisico.equals(copia));
System.out.println(fisico.equals(ebook));
System.out.println(fisico.equals("978-85-5519-023-1"));
System.out.println(fisico.equals(null));
CarrinhoDeCompras outra = new CarrinhoDeCompras();
outra.adiciona(fisico);
outra.remove(copia);
System.out.println(outra.getQuantidade());
String a = "livro";
String b = new String("livro");
System.out.println(a == b);
System.out.println(a.equals(b));
System.out.println(ebook.getClass().getSimpleName());
```

Etapa 2: saída esperada

```
--- Etapa 2 ---
false
true
false
false
false
0
false
true
Ebook
```

- `fisico` e `copia` são objetos diferentes (== é `false`), mas têm o mesmo ISBN
- O `remove(copia)` removeu o `fisico`: a quantidade caiu para `0`

Um Ebook **é um** Livro

```
1 Ebook e1 = new Ebook(autor);
2 Livro e2 = new Ebook(autor);       // vale: todo Ebook e um Livro
```

- Não estamos **transformando** o objeto: ele continua sendo um Ebook
- Só mudamos **como nos referimos** a ele
- Pela referência Livro, só se enxerga o que Livro tem:

```
1 System.out.println(e2.getWaterMark());
```

```
error: cannot find symbol
  symbol:   method getWaterMark()
  location: variable e2 of type Livro
```

Qual método executa?

```
1 Livro livro = new Ebook(autor);
2 livro.setValor(30.00);
3 System.out.println(livro.aplicaDescontoDe(0.2));
```

```
Desconto acima do permitido para ebook
false
```

- O tipo da **referência** é `Livro`; o tipo do **objeto** é `Ebook`
- Em execução, vale a regra do **objeto**: a do `Ebook`
- Isso é o **polimorfismo**: a mesma chamada, comportamentos diferentes conforme o objeto

Um método para todos os livros

Sem polimorfismo, um `adiciona` para cada tipo:

```
1 public void adiciona(Ebook livro) { ... }
2 public void adiciona(LivroFisico livro) { ... }
3 public void adiciona(Audiolivro livro) { ... }   // e a cada tipo novo...
```

Com polimorfismo, um só:

```
1 public void adiciona(Livro livro) { this.livros.add(livro); }
```

- O `CarrinhoDeCompras` já é assim: `List<Livro>` guarda os três tipos
- Um novo tipo de livro **não obriga** a mexer no carrinho

Casting: quando preciso do tipo da filha

```java
Livro livro = new Ebook(autor);
if (livro instanceof Ebook) {
    Ebook ebook = (Ebook) livro;        // casting
    System.out.println(ebook.getWaterMark());
}
```

Sem o teste, um tipo errado quebra **em execução**:

```java
Livro livro = new LivroFisico(autor);
Ebook ebook = (Ebook) livro;
```

```
Exception in thread "main" java.lang.ClassCastException: class ...LivroFisico cannot be cast to class ...Ebook
```

- `instanceof` pergunta "este objeto é desse tipo?" antes do *cast*
- Casting demais é sinal de alerta: o polimorfismo existe para **evitar** isso

Etapa 3: objetivo

No `CarrinhoDeCompras`:

1. `int aplicaDescontoEmTodos(double porcentagem)`: pede o desconto a **cada** livro, sem saber o tipo dele, e devolve **quantos aceitaram**
2. `double getTotalDeTaxasDeImpressao()`: soma a taxa de impressão **só** dos `LivroFisico`

No `main`: veja o desconto do `Ebook` sendo aplicado por uma referência `Livro`.

- Reaproveite os objetos `ebook`, `fisico` e `audio` da etapa 1 (o `ebook` já tem valor `27.0`)

Etapa 3: pistas

- No primeiro método, **nenhum** `instanceof`: é o polimorfismo que escolhe a regra
- O segundo é o caso que **precisa** de casting: `getTaxaImpressao` só existe em `LivroFisico`
- Percorra `this.livros` com *enhanced-for*
- `aplicaDescontoDe` já devolve `boolean`: conte os `true`
- Imprima a taxa com `String.format("R$ %.2f", ...)`

Etapa 3: checkpoint

```
1  System.out.println("--- Etapa 3 ---");
2  CarrinhoDeCompras carrinho = new CarrinhoDeCompras();
3  carrinho.adiciona(ebook);
4  carrinho.adiciona(fisico);
5  carrinho.adiciona(audio);
6  System.out.println(carrinho.getQuantidade());
7  System.out.println(carrinho.aplicaDescontoEmTodos(0.2));
8  System.out.println(carrinho.getTotal());
9  System.out.println(String.format("R$ %.2f",
10         carrinho.getTotalDeTaxasDeImpressao()));
11 Livro generico = new Ebook(aniche);
12 generico.setValor(30.00);
13 System.out.println(generico.aplicaDescontoDe(0.2));
14 if (generico instanceof Ebook) {
15     Ebook convertido = (Ebook) generico;
16     convertido.setWaterMark("teste");
17     System.out.println(convertido.getWaterMark());
18 }
```

Etapa 3: saída esperada

```
--- Etapa 3 ---
3
Desconto acima do permitido para ebook
2
107.0
R$ 2,40
Desconto acima do permitido para ebook
false
teste
```

- O desconto de 20% foi **recusado** pelo `Ebook` (limite de 15%) e aceito pelos outros dois: $27 + 48 + 32 = 107$
- A taxa é de 5% sobre os `R$ 48,00` do livro físico já com desconto

O que é um Livro?

```
1 Livro livro = new Livro(autor);     // e-book? fisico? audio?
```

- Um `Livro` puro é só uma **abstração**: o que todos os tipos têm em comum
- Nunca deveríamos **vender** um `Livro`, mas sim uma de suas filhas

Solução: o modificador `abstract`.

```
1 public abstract class Livro { ... }
```

```
error: Livro is abstract; cannot be instantiated
```

A classe abstrata ainda é útil

- Não dá para **instanciar**, mas dá para usar como **referência**:

```
1 Livro livro = new LivroFisico(autor);          // vale
2 List<Livro> livros = new ArrayList<>();         // vale
3 public void adiciona(Livro livro) { ... }       // vale
```

- Ela concentra atributos e comportamentos comuns, e serve para **herança** e **polimorfismo**
- Todo código que fazia `new Livro(...)` precisa passar a criar uma **filha**
- Quando ser abstrata? Quando a classe é tão genérica que não existe "um qualquer" dela (o que seria um `Animal` puro?)

Método abstrato

E se cada tipo precisa ter uma resposta própria? O método abstrato **só declara**:

```java
public abstract class Livro {
    public abstract String getFormato();      // sem corpo
}

public class Ebook extends Livro {
    @Override
    public String getFormato() {
        return "Ebook";
    }
}
```

```
error: Audiolivro is not abstract and does not override abstract method
    getFormato() in Livro
```

Toda filha **concreta** é obrigada a implementar: esquecer vira erro de compilação.

Regras da classe abstrata

- Pode ser abstrata **sem** nenhum método abstrato — mas então ninguém a instancia
- Se tem método abstrato, a classe **precisa** ser abstrata
- Pode ter métodos abstratos **e** concretos (com corpo)
- Toda filha implementa os métodos abstratos — a não ser que também seja abstrata
- Não pode haver `new` de uma classe abstrata; construtores continuam servindo, chamados com `super(...)`

Etapa 4: objetivo

1. Torne `Livro` **abstract**. O projeto deixa de compilar: veja o erro e corrija o `main`, criando `LivroFisico` onde havia `new Livro(...)`
2. Em `Livro`, crie `public abstract String getFormato()`
3. Implemente em cada filha, com `@Override`:
   - `Ebook` → `"Ebook"`
   - `LivroFisico` → `"Livro fisico"`
   - `Audiolivro` → `"Audiolivro"`

Experimente **apagar** o método do `Audiolivro` e ler a mensagem do compilador; depois devolva.

Etapa 4: pistas

- Dois `new Livro(...)` no começo do `main`: desbravando e cabeca
- A variável pode continuar do tipo `Livro`: `Livro l = new LivroFisico(autor);`
- `toString`, `equals` e os *getters* continuam na `Livro`: só o `getFormato` é abstrato
- O `Catalogo` e o `CarrinhoDeCompras` **não** mudam

Etapa 4: checkpoint

Acrescente ao final do `main` e rode:

```
System.out.println("--- Etapa 4 ---");
for (Livro item : carrinho.getLivros()) {
    System.out.println(item.getFormato());
}
```

```
--- Etapa 4 ---
Ebook
Livro fisico
Audiolivro
```

- A saída de **todas** as etapas anteriores continua a mesma

Desafio bônus: `calculaFrete()`

Mais um método abstrato em `Livro`: `public abstract double calculaFrete()`

- `Ebook` e `Audiolivro`: `0.0` (são baixados)
- `LivroFisico`: valor fixo, constante `VALOR_DO_FRETE = 12.50`
- `CarrinhoDeCompras`: `double getTotalComFrete()` = total + frete de cada livro

```
1 System.out.println("--- Bonus ---");
2 System.out.println(ebook.calculaFrete());
3 System.out.println(fisico.calculaFrete());
4 System.out.println(carrinho.getTotalComFrete());
```

```
--- Bonus ---
0.0
12.5
119.5
```

Exit ticket

Em uma folha, com o nome da dupla:

1. O que acontece se você escrever `@Override` num método com o nome errado? Por que isso ajuda?
2. Por que `fisico == copia` é `false` e `fisico.equals(copia)` é `true`?
3. Em `Livro l = new Ebook(autor)`, qual é o tipo da referência e qual é o tipo do objeto? Qual `aplicaDescontoDe` executa?
4. Qual a diferença entre um método **abstrato** e um método **sobrescrito**?

Resumo

- **Etapa 1:** sobrescrita, `@Override`, `super.metodo()`, `toString`
- **Etapa 2:** todo objeto é um `Object`; `equals` $\times$ `==`; `String` sobrescreve `equals`
- **Etapa 3:** `Livro l = new Ebook(...)`, polimorfismo, `instanceof` e casting
- **Etapa 4:** `abstract class`, método abstrato, filhas obrigadas a implementar

Entrega: o projeto em `.zip` no Moodle, no prazo indicado.

Próxima aula

- `Ebook` e `Audiolivro` podem ser **baixados**; `LivroFisico` não
- Um método `baixar()` na `Livro` seria uma mentira para o livro físico
- Na semana 12: **interfaces**, um contrato que classes sem parentesco podem assinar

Referências

SCHILDT, Herbert. **Java**: a beginner's guide. 9. ed. Nova York: McGraw Hill, 2022. (cap. 7)
TURINI, Rodrigo. **Desbravando Java e Orientação a Objetos**: um guia para o iniciante da linguagem. [S.l.: s.n.], [20–]. (seções 5.2, 5.4, 5.5, 6.1 a 6.4, 10.1 e 10.3)